

Deep Learning Browser Open Source Frameworks

Table 1

Platform	Description
TensorFlow.js	This is Google's open source initiative released in March 2018. So far, it is the best in the breed. It consists of various components. TensorFlow.js supports WebGL, a browser interface of OpenGL. This means that if the GPU is available, it will accelerate using it, otherwise it will fall back to the CPU. More details are available at https://github.com/tensorflow/tfjs
Keras.js	The semantic is exactly similar to that of TensorFlow in Python. More details at https://github.com/transcranial/keras-js
WebDNN	This is from the University of Tokyo. It is the fastest DNN execution framework in a browser. Details are at https://github.com/mil-tokyo/webdnn
ConvNetJS	ConvNetJS is a JavaScript library for training deep learning models (neural networks) entirely in your browser. It is no longer maintained. Details are at https://cs.stanford.edu/people/karpathy/convnetjs/
Brain.js	This is a GPU accelerated neural network in JavaScript for browsers and Node.js. Details at https://github.com/BrainJS
Mind	This is a flexible neural network library for Node.js and the browser (https://github.com/stevenmiller888/mind).
Synaptic	Synaptic is a JavaScript neural network library for Node.js and the browser. Its generalised algorithm is architecture-free, so you can build and train basically any type of first order or even second order neural network architectures (https://github.com/cazala/synaptic).

```
<p id="prediction">Predicted Value:
</p>
<script>
  async function do_prediction(){
    const model=tf.sequential();
    model.add(
      tf.layers.dense({
        units:1,
        inputShape:[1],
        bias: true
      })
    );
    model.compile({
      loss:'meanSquaredError',
      optimizer: 'sgd',
      metrics: ['mse']
    });
    const train_labels = tf.tensor1d
    ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]);
    const train_output = tf.tensor1d
    ([2, 5, 8, 12, 14, 18, 21, 23, 26, 29]);
    await model.fit(train_labels,
    train_output, {epochs:100});
    const inputVal = parseInt(document.
    getElementById('inputNumber').value);
    document.getElementById
    ('prediction').innerText += model.
    predict(tf.tensor1d([inputVal])).data
    Sync();
  }
</script>
</body>
</html>
```

Let us understand this code a bit. The first part of the code is purely the HTML code for the input textbox, the *Predict* button, and the output section. The 'script' portion is where the TensorFlow.js magic works. It just creates a very simple, linear regression sort of one-layer sequential dense model.

```
const model=tf.sequential();
model.add(
  tf.layers.dense({
```

```
units:1,
inputShape:[1],
bias: true
});
```

The model is built with SGD (stochastic descent gradient) as optimiser, MSE (mean square error) as loss function, and the accuracy measurement yardstick. This code looks very similar to the familiar Python Keras code.

```
model.compile({
  loss:'meanSquaredError',
  optimizer: 'sgd',
  metrics: ['mse']
});
```

Next, we need to get the training data set ready. We are training with 1 to 10 as input (x values) and input * 3 -1 as output (y values):

```
const train_labels = tf.tensor1d([1, 2, 3,
4, 5, 6, 7, 8, 9, 10]);
const train_output = tf.tensor1d([2, 5, 8,
12, 14, 18, 21, 23, 26, 29]);
```

Now, let's do some training with the 'fit' method of TensorFlow layer's API:

```
await model.fit(train_labels, train_output,
{epochs:100});
```

Next, let's have the prediction, by getting the input text the user has given. Note the TensorFlow.js layer's 'model.predict' API (there's no change when compared to Python Keras!):

```
const inputVal = parseInt(document.
getElementById('inputNumber').value);
document.getElementById('prediction')
.innerText +=model.predict(tf.tensor1d
([inputVal])).dataSync();
```

In the end, the 'do_prediction()' method is invoked on clicking the

TensorFlow JS example. Input a number and get prediction near to (3*input)-1.

Enter an integer here (Beyond 1 to 10)

Predicted Value:34.72138214111328

Fig. 1: The demo program output

Predict button:

```
<button onclick="do_prediction()
">Predict</button>
```

And the output in your favourite browser is something like what's shown in Fig. 1.

Takeaway

It is an accepted fact today that the fully connected neural network based inference works perfectly in browsers. Typically, it is not the training but the inference that happens in the browser. Keeping in mind the fact that ConvNetJS is not maintained any more, Google's TensorFlow.js is the best bet in this regard. It is not necessary that the browser's host must be GPU accelerated, because the sort of inference that runs on a browser does not fully exploit the GPU. Hence, running it in a 'normal laptop' is good enough as far as the inference task in the browser is concerned. **EFY**

This article was first published in March 2021 issue of Open Source For You



Pradip Mukhopadhyay has 20 years of experience across the stack—from low level systems programming to high level GUIs. He is a FOSS enthusiast, and currently works at NetApp, Bengaluru